

---

# ***Ejercicios JAVA***

*Ejercicios Guía*

Hecho por: Matías González

---

# Índice

---

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>Índice.....</b>                                        | <b>2</b>  |
| <b>Ejercicio 1: TechMarine (Tipo Prueba).....</b>         | <b>3</b>  |
| – Contexto –.....                                         | 3         |
| – Información necesaria –.....                            | 3         |
| – Métodos especializados –.....                           | 4         |
| – Validaciones esperadas –.....                           | 5         |
| – Operaciones del sistema –.....                          | 5         |
| – Diagrama de Clases –.....                               | 6         |
| – Resultado esperado –.....                               | 7         |
| <b>Ejercicio 2: Hotel Bosques de Viña (Medio).....</b>    | <b>8</b>  |
| – Contexto –.....                                         | 8         |
| – Objetivos –.....                                        | 8         |
| – Diagrama de Clases –.....                               | 16        |
| <b>Ejercicio 3: Registro Planetario II (Difícil).....</b> | <b>17</b> |
| – Contexto –.....                                         | 17        |
| – Objetivos –.....                                        | 18        |
| – Consideraciones –.....                                  | 18        |
| – El sistema deberá validar–.....                         | 19        |
| – Modelo de clases –.....                                 | 21        |
| <b>¡Ánimo Chicos!.....</b>                                | <b>26</b> |

---

# Ejercicio 1: TechMarine (Tipo Prueba)

---

## – Contexto –

La tienda de computadores **TechMarine** requiere un sistema para administrar los computadores que tiene disponibles para la venta y calcular el precio final de cada uno.

La tienda trabaja con distintos tipos de computadores; aunque todos comparten información básica como **marca**, **modelo** y **precio base**, el precio final varía según el tipo de computador.

Adicionalmente, ciertos computadores pueden participar en un programa de **garantía extendida**, lo que implica gestionar su disponibilidad de forma diferenciada.

El sistema debe modelar esta situación aplicando los principios de la Programación Orientada a Objetos, de manera que el comportamiento compartido y el comportamiento específico de cada tipo de computador queden correctamente organizados en la jerarquía de clases.

## – Información necesaria –

El sistema contempla las entidades que se muestran en el diagrama. Cada clase debe incluir:

- Constructor(es)
- Getters
- Setters
- `toString()`

El método `toString()` debe incluir **únicamente** la **marca** y **modelo** del computador.

Los constructores deben delegar la asignación de valores a los métodos setter, de modo que las validaciones definidas se apliquen desde la construcción del objeto.

## Computadores

Todos los computadores comparten:

- **Marca**
- **Modelo**
- **Precio base**

*Dado que el precio final se determina de manera diferente para cada tipo de computador, el sistema debe contemplar un mecanismo que permita calcular ese precio de forma general sin conocer el tipo específico del objeto en cada momento.*

---

## Notebooks

Los notebooks agregan:

- **Tamaño** de pantalla
- Un **indicador** de si poseen garantía extendida

Los notebooks que participan en el programa de garantía deben implementar el contrato definido por la interfaz **Garantizable**.

## Computadores de escritorio

Los computadores de escritorio agregan:

- **Tipo** de gabinete
- Un **indicador** de si incluyen monitor

## Gestor (Colección)

La clase **Tienda** es responsable de administrar la colección de computadores registrados y exponer las operaciones sobre ella.

## – Métodos especializados –

El sistema debe ser capaz de calcular el **precio final** para cualquier computador registrado, independientemente de su tipo.

Cada tipo determina ese precio según su propia lógica.

### Notebook

- El precio base es el precio ingresado.
- Si posee garantía extendida, el precio aumenta en un **10%**.

### Computador de escritorio

- El precio base es el precio ingresado.
- Si incluye monitor, se agregan **\$100.000**.

## Interfaz **Garantizable**

La interfaz **Garantizable** define el contrato que deben cumplir los computadores que participan en el programa de garantía.

---

Está compuesta por dos operaciones:

- La primera permite consultar si el computador tiene garantía extendida.
- La segunda permite registrar que el computador tendrá garantía extendida.

Esta interfaz está diseñada de manera que, en el futuro, cualquier tipo de computador pueda incorporarse al programa sin necesidad de modificar la jerarquía de clases existente.

Por ahora, **solo los notebooks implementan este contrato**.

### – *Validaciones esperadas* –

Las validaciones deben aplicarse en los métodos setter de cada clase. Cuando una condición no se cumpla, con un mensaje descriptivo que explique el motivo del rechazo, debe lanzarse una excepción de tipo:

`IllegalArgumentException`

#### Reglas:

- **Marca:** No puede ser nula ni vacía.
- **Modelo:** No puede ser nulo ni vacío.
- **Precio base:** Debe ser mayor que cero.
- **Tamaño de pantalla:** Debe ser mayor que cero.
- **Tipo de gabinete:** No puede ser nulo ni vacío.

### – *Operaciones del sistema* –

La clase `Tienda`, que gestiona la colección de computadores, debe exponer las siguientes operaciones:

**Registrar un computador:** Debe agregar el computador a la colección e informar por consola que fue incorporado correctamente.

**Buscar por marca:** Debe buscar y retornar los computadores cuya marca coincida con el criterio de búsqueda recibido.

#### Requerimientos de la clase principal

La clase principal debe demostrar el funcionamiento completo del sistema e incluir los siguientes elementos:

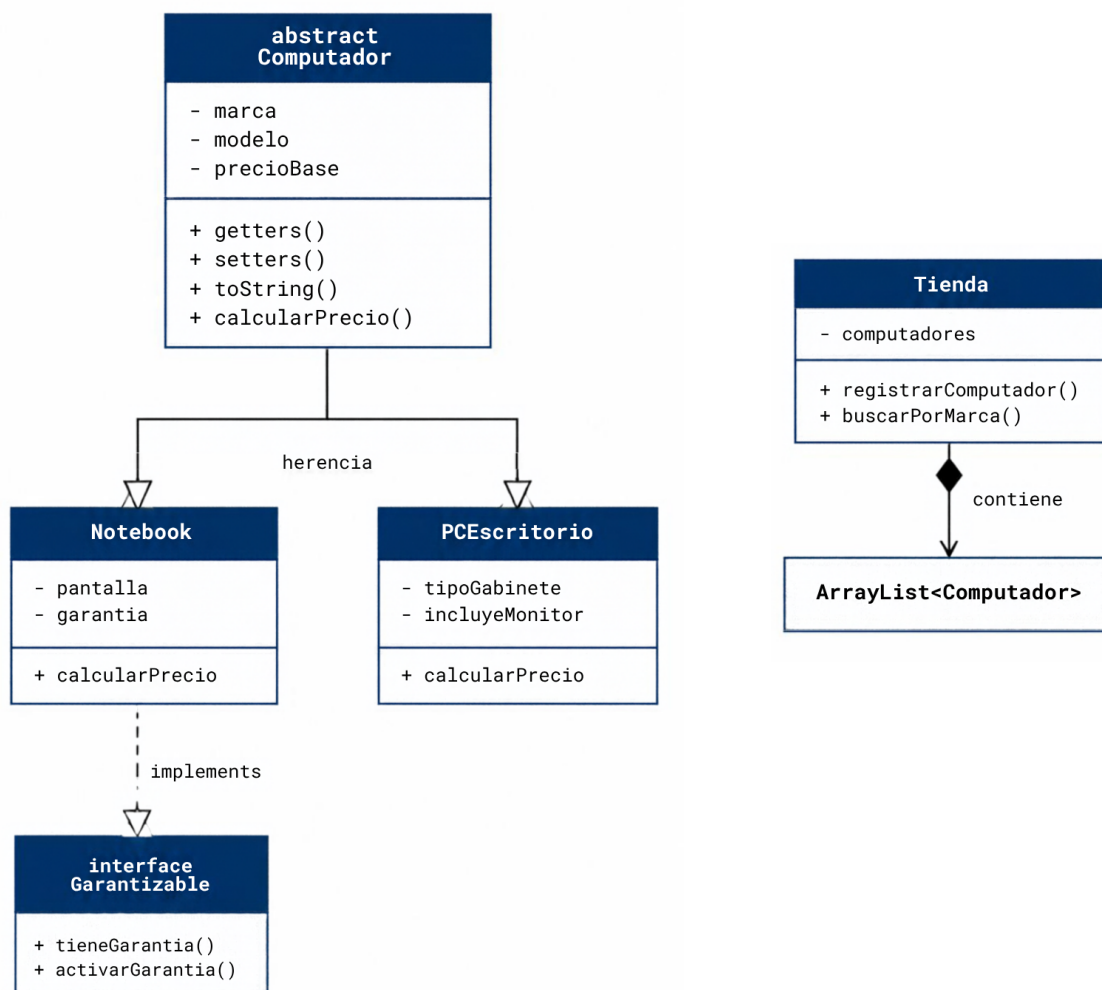
- Instanciar un computador de cada tipo disponible utilizando los datos de la tabla.
- Marcar el notebook **Lenovo IdeaPad 3** con garantía extendida una vez creado.

- Registrar todos los computadores en la tienda.
- Mostrar el precio final de cada computador.
- Buscar computadores por marca.
- Listar todos los computadores registrados mediante el método `toString()` de cada objeto.

### Datos para la clase principal

| Tipo          | Marca  | Modelo    | Precio base | Atributo específico            |
|---------------|--------|-----------|-------------|--------------------------------|
| Notebook      | Lenovo | IdeaPad 3 | \$800.000   | Pantalla: 15.6"                |
| Notebook      | HP     | Pavilion  | \$900.000   | Pantalla: 14"                  |
| PC Escritorio | Asus   | Gamer X   | \$1.000.000 | Gabinete: Gamer / Monitor: Sí  |
| PC Escritorio | Dell   | Office    | \$600.000   | Gabinete: Normal / Monitor: No |

### – Diagrama de Clases –



---

## – Resultado esperado –

Se aceptan pequeñas diferencias en los textos o en el formato de los números, siempre que se muestren los datos solicitados y los resultados numéricos sean correctos. **La ejecución del programa debe mostrar información similar a:**

```
Computador Lenovo IdeaPad 3 registrado correctamente.
Computador HP Pavilion registrado correctamente.
Computador Asus Gamer X registrado correctamente.
Computador Dell Office registrado correctamente.

PRECIOS FINALES

Lenovo IdeaPad 3: $880.000
HP Pavilion: $900.000
Asus Gamer X: $1.100.000
Dell Office: $600.000

BUSQUEDA POR MARCA: Lenovo

Lenovo - IdeaPad 3

TODOS LOS COMPUTADORES

Lenovo - IdeaPad 3
HP - Pavilion
Asus - Gamer X
Dell - Office
```

---

# Ejercicio 2: Hotel Bosques de Viña (Medio)

---

## – Contexto –

El **Hotel Bosques de Viña** necesita desarrollar un sistema básico para gestionar las reservas realizadas por sus huéspedes.

Actualmente el hotel maneja distintos tipos de reservas. Algunas corresponden a reservas de habitaciones realizadas por huéspedes particulares y otras corresponden a reservas corporativas realizadas por empresas.

Para organizar el sistema se requiere utilizar **Programación Orientada a Objetos**, aplicando conceptos de **clases abstractas, herencia, interfaces, encapsulamiento, polimorfismo, sobrescritura de métodos, validaciones y manejo de excepciones**.

El sistema deberá permitir registrar reservas, calcular sus costos de manera polimórfica, buscar reservas y mostrar la información registrada.

## – Objetivos –

### 1. Creación de la clase abstracta **Reserva**

Cree una clase abstracta llamada **Reserva**. La clase debe contener los siguientes atributos privados:

- `nombreHuesped`
- `numeroNoches`
- `precioPorNoche`
- `cantidadHuespedes`

Además, debe contener:

- Constructor.
- Getters y setters para todos sus atributos.
- Método `toString()`.
- Método abstracto `calcularCostoTotal()`.

El método abstracto debe tener la siguiente estructura:

```
public abstract double calcularCostoTotal();
```

El constructor debe utilizar los **setters** para asignar los valores recibidos.

---

El método `toString()` debe mostrar al menos:

- Nombre del huésped.
- Cantidad de noches.

## 2. Validaciones de la clase `Reserva`

Los setters deben validar los datos recibidos. Si un dato no cumple la condición indicada, se debe lanzar una excepción:

`IllegalArgumentException`.

- `nombreHuesped`: No puede ser null ni estar vacío.
- `numeroNoches`: Debe ser mayor que 0.
- `precioPorNoche`: Debe ser mayor que 0.
- `cantidadHuespedes`: Debe ser mayor que 0.

Por ejemplo:

```
if (numeroNoches <= 0) {  
    throw new IllegalArgumentException("El número de noches debe ser  
    mayor que 0");  
}
```

## 3. Clase `ReservaHabitacion`

Cree una clase llamada `ReservaHabitacion` que herede de `Reserva`. Debe agregar los siguientes atributos privados:

- `tipoHabitacion`
- `incluyeDesayuno`

El atributo `tipoHabitacion` solamente puede aceptar los siguientes valores:

- `"Simple"`
- `"Doble"`
- `"Suite"`

Además, debe crear:

- Constructor.
- Getters y setters.
- Sobreescritura de `calcularCostoTotal()`.

- 
- Sobreescritura de `toString()`.

### *Cálculo del costo de una reserva de habitación*

El costo base corresponde al '`precioPorNoche`' por '`numeroNoches`'. También se deben aplicar las siguientes reglas dependiendo de la reserva:

- Si el tipo de habitación es "`Suite`", se agrega un **30%** al costo base.
- Si la reserva incluye "`desayuno`", se agrega **\$8.000 por noche**.

El cálculo debe realizarse considerando ambas condiciones cuando corresponda. Por ejemplo, una reserva de una Suite con desayuno debe aplicar:

1. Precio de las noches.
2. Recargo del 30% por Suite.
3. Costo del desayuno por cada noche.

## 4. Validación de `tipoHabitacion`

El setter de `tipoHabitacion` debe verificar que solamente se acepten:

- Simple
- Doble
- Suite

Si se intenta ingresar otro valor, debe lanzar: `IllegalArgumentException`

## 5. Interfaz `CheckInAnticipado`

Cree una interfaz llamada '`CheckInAnticipado`'. La interfaz debe declarar los siguientes métodos:

- `boolean tieneCheckInAnticipado();`
- `void solicitarCheckInAnticipado();`

La clase `ReservaHabitacion` debe implementar esta interfaz.

Debe agregar en `ReservaHabitacion` un atributo que permita determinar si el huésped tiene solicitado el check-in anticipado.

**Se deben implementar los métodos:**

- 
- `solicitarCheckInAnticipado()`: debe modificar dicho estado.
  - `tieneCheckInAnticipado()`: debe permitir consultar el estado actual.

## 6. Clase `ReservaCorporativa`

Cree una clase llamada `ReservaCorporativa` que herede de `Reserva`. Debe agregar los siguientes atributos privados:

- `nombreEmpresa`
- `tieneConvenio`

Debe implementar:

- Constructor.
- Getters y setters.
- Sobreescritura de `calcularCostoTotal()`.
- Sobreescritura de `toString()`.

### *Cálculo del costo de una reserva corporativa*

El costo inicial corresponde al '`precioPorNoche`' por '`numeroNoches`'. Luego se deben aplicar las siguientes condiciones:

- **Si la empresa tiene convenio con el hotel:** 15% de descuento
- **Si la reserva tiene más de 5 huéspedes:** 10% de recargo

**Ambas condiciones pueden aplicarse al mismo tiempo.**

## 7. Validación de `nombreEmpresa`

El setter de `nombreEmpresa` debe validar que el nombre:

- No sea `null`.
- No esté vacío.

Si no cumple la condición, debe lanzar:

`IllegalArgumentException`

## 8. Polimorfismo

El sistema debe demostrar el uso de **polimorfismo**. Para esto, se deben poder almacenar objetos de tipo:

- 
- ReservaHabitacion
  - ReservaCorporativa

dentro de una colección de tipo:

```
ArrayList<Reserva>
```

Luego, se debe recorrer la colección y utilizar:

```
calcularCostoTotal()
```

Sin necesidad de preguntar qué tipo específico de reserva es. **Por ejemplo:**

```
for (Reserva reserva : reservas) {  
    System.out.println(reserva.calcularCostoTotal());  
}
```

El método que se ejecutará dependerá del objeto almacenado.

## 9. Clase **GestorReservas**

**Cree una clase llamada:** **GestorReservas**. Esta clase será responsable de administrar las reservas del hotel.

**Debe contener un atributo:**

```
ArrayList<Reserva>
```

**El gestor debe implementar los siguientes métodos:**

### ***a. Registrar una reserva***

Debe agregar la reserva a la lista.

```
public void registrarReserva(Reserva reserva)
```

### ***b. Buscar por huésped***

Debe recorrer la lista y buscar una reserva cuyo nombre de huésped coincida con el nombre recibido.

```
public Reserva buscarPorHuesped(String nombre)
```

- Si encuentra una coincidencia, debe retornar la reserva.
- Si NO encuentra ninguna, debe retornar: `null`

### c. Listar reservas

Debe recorrer todas las reservas registradas y mostrar su información. Se debe aprovechar el polimorfismo para mostrar también el costo total de cada reserva.

```
public void listarReservas()
```

## 10. Clase `Main`

Cree una clase `Main` que permita probar el funcionamiento completo del sistema. Debe crear las siguientes reservas.

| N° Reserva | Tipo de Reserva | Información de reserva                                                                                                                                                                                                         |
|------------|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1          | Habitación      | <ul style="list-style-type: none"> <li>• Huésped: Juan Pérez</li> <li>• Noches: 3</li> <li>• Precio por noche: \$50.000</li> <li>• Huéspedes: 2</li> <li>• Tipo habitación: Suite</li> <li>• Incluye desayuno: Sí</li> </ul>   |
| 2          | Habitación      | <ul style="list-style-type: none"> <li>• Huésped: María Soto</li> <li>• Noches: 2</li> <li>• Precio por noche: \$40.000</li> <li>• Huéspedes: 1</li> <li>• Tipo habitación: Simple</li> <li>• Incluye desayuno: No</li> </ul>  |
| 3          | Corporativa     | <ul style="list-style-type: none"> <li>• Huésped: Pedro González</li> <li>• Noches: 5</li> <li>• Precio por noche: \$70.000</li> <li>• Huéspedes: 8</li> <li>• Empresa: TechSolutions</li> <li>• Tiene convenio: Sí</li> </ul> |
| 4          | Corporativa     | <ul style="list-style-type: none"> <li>• Huésped: Ana Torres</li> <li>• Noches: 4</li> <li>• Precio por noche: \$60.000</li> <li>• Huéspedes: 4</li> <li>• Empresa: Innova Ltda.</li> <li>• Tiene convenio: No</li> </ul>      |

---

## 11. Registro de las reservas

Una vez creadas las reservas, debe crear un objeto de:

```
GestorReservas
```

y registrar las cuatro reservas.

## 12. Prueba del CheckInAnticipado

Para la reserva de **Juan Pérez**:

1. Solicite el check-in anticipado.
2. Consulte si tiene check-in anticipado.
3. Muestre el resultado por consola.

Por ejemplo:

```
Juan Pérez tiene check-in anticipado: true
```

## 13. Listado de reservas

Para mostrar todas las reservas registradas, se debe Utilizar el método:

```
listarReservas()
```

Para cada reserva se debe mostrar información similar a:

```
Huésped: Juan Pérez  
Noches: 3  
Costo total: $...
```

El costo debe calcularse mediante:

```
calcularCostoTotal()
```

## 14. Búsqueda de reservas

Debe mostrar la reserva encontrada mediante un nombre Usando el método:

```
buscarPorHuesped()
```

**Luego**, realice una segunda búsqueda utilizando un huésped que **no exista**. Debe controlar el caso en que el método retorna `null`.

---

## 15. Prueba de excepciones

**Finalmente**, pruebe las validaciones implementadas en los setters. Debe realizar al menos las siguientes pruebas:

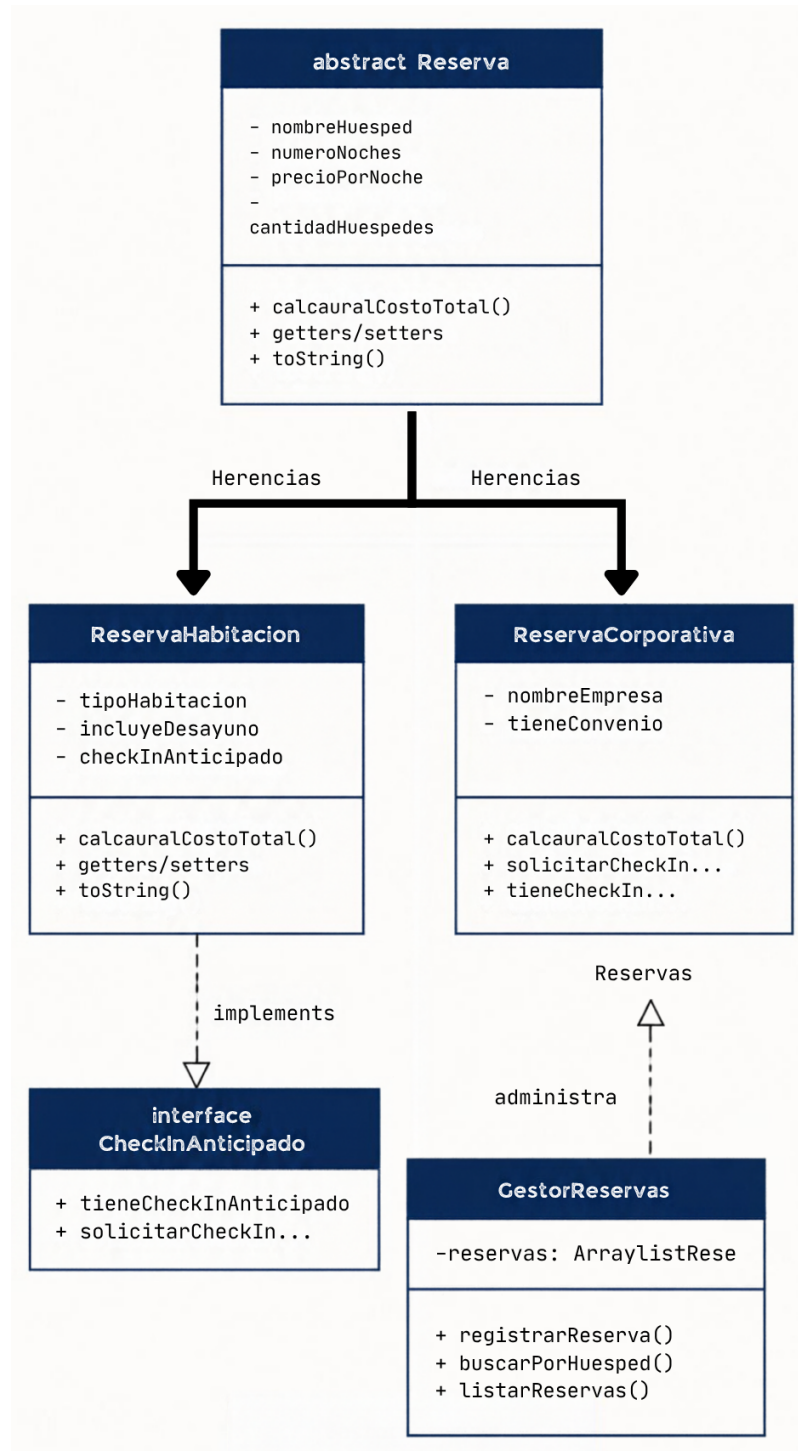
1. Intentar crear una reserva con: `'numeroNoches = 0'`
2. Intentar crear una reserva con: `'precioPorNoche = -10000'`

Las excepciones deben ser controladas utilizando `try-catch`. Ejemplo:

```
try {  
    } catch (IllegalArgumentException e) {  
        System.out.println(e.getMessage());  
    }  
}
```

## – Diagrama de Clases –

Se les entrega un Diagrama de Clases para ayudar a entender la solución esperada.



# Ejercicio 3: Registro Planetario II (Difícil)

## – Contexto –

En el milenio 41, la Super Tierra continúa expandiendo su influencia a través de miles de sistemas estelares. Cada nuevo mundo descubierto debe ser registrado y clasificado según sus características demográficas para garantizar una correcta administración de recursos, defensa y producción. La Oficina de Colonización Planetaria ha solicitado el desarrollo de un sistema que permita almacenar información sobre los planetas bajo control de la Super Tierra.

Cada planeta deberá contar con:

- Un código planetario único.
- Un nombre.
- Una cantidad de habitantes.
- Una clasificación planetaria.
- Un estado de colonización.

Para representar correctamente los diferentes tipos de planetas, el sistema deberá utilizar Programación Orientada a Objetos, aplicando clases, clases abstractas, herencia, polimorfismo, interfaces y colecciones.

La clasificación de cada planeta dependerá exclusivamente de su población. El hecho de que un planeta se encuentre colonizado o no será representado mediante una especialización de la clase `Planeta`, pero no determinará su clasificación planetaria.

La siguiente tabla muestra los tipos de mundos reconocidos oficialmente y los rangos poblacionales asociados a cada uno de ellos:

| Tipo       | Descripción del planeta                                                                               | Cantidad Min | Cantidad Máx |
|------------|-------------------------------------------------------------------------------------------------------|--------------|--------------|
| Colmena    | Planetas-ciudad masivamente superpoblados, caracterizados por imponentes torres y niveles inferiores. | 100.000.001  | $\infty+$    |
| Forja      | Dedicados exclusivamente a la manufactura, cubiertos por completo de maquinaria industrial.           | 100.000.001  | $\infty+$    |
| Agrícolas  | Planetas rurales utilizados exclusivamente para producir alimentos para el resto de los sistemas.     | 100.000.001  | $\infty+$    |
| Mortíferos | Planetas con entornos extremadamente hostiles, a menudo letales, y ecosistemas agresivos.             | 1            | 100.000.000  |
| Salvajes   | Mundos preindustriales o de nivel medieval.                                                           | 1            | 100.000.000  |
| Muertos    | Planetas estériles, desolados o destruidos.                                                           | 0            | 0            |

Deberás desarrollar una aplicación en Java que permita registrar, consultar, actualizar y eliminar información de las colonias registradas, asegurando que cada planeta sea clasificado correctamente según las reglas establecidas por la administración central de la Super Tierra.

---

***“Ser el primero en entrar en el cosmos, entablar, en solitario, un duelo sin precedentes con la naturaleza... ¿se puede soñar con algo más?”***

**– Yuri Gagarin, el primer ser humano en el espacio. –**

### ***– Objetivos –***

Desarrollar una aplicación de escritorio en **Java** que permita administrar el registro de planetas pertenecientes a la Super Tierra, aplicando los principales conceptos de **Programación Orientada a Objetos**.

**El sistema deberá permitir:**

- **Registrar** nuevos planetas.
- **Buscar información** de un planeta mediante su código.
- **Actualizar** la población de un planeta.
- **Recalcular** su clasificación al actualizar la población.
- **Eliminar** planetas del registro.
- **Mostrar** todos los planetas registrados.
- **Mantener el sistema en ejecución** mediante un menú interactivo hasta seleccionar la opción de salida.

### ***– Consideraciones –***

**Cada planeta deberá almacenar la siguiente información:**

- **Código** planetario.
- **Nombre** del planeta.
- **Población**.
- **Tipo de Planeta**.

**Los datos deberán almacenarse utilizando ArrayList en Java y el código planetario será único para cada planeta.**

**Los tipos de mundos válidos serán:**

- **Colmena**
- **Forja**
- **Agrícola**
- **Mortífero**
- **Muerto**
- **Salvaje**

---

La población determinará los posibles tipos de planeta que pueden ser asignados:

| Tipo Planeta | Cantidad Min | Cantidad Máx |
|--------------|--------------|--------------|
| Colmena      | 100.000.001  | $\infty+$    |
| Forja        | 100.000.001  | $\infty+$    |
| Agrícolas    | 100.000.001  | $\infty+$    |
| Mortíferos   | 1            | 100.000.000  |
| Salvajes     | 1            | 100.000.000  |
| Muertos      | 0            | 0            |

- Si la población es igual a 0, el tipo de planeta deberá asignarse **automáticamente como "Muerto"**.
- Si la población está entre 1 y 100.000.000 habitantes, el usuario deberá seleccionar entre los tipos **"Mortífero"** y **"Salvaje"**.
- Si la población es superior a 100.000.000 habitantes, el usuario deberá seleccionar entre los tipos **"Colmena"**, **"Forja"** y **"Agrícola"**.

Al actualizar la población de un planeta, el tipo deberá de re-calcularse y actualizarse según las reglas anteriores.

### ***– El sistema deberá validar–***

**El sistema deberá validar correctamente los datos ingresados por el usuario.**

Se deberá controlar:

- Que las opciones del menú sean válidas.
- Que las opciones de **clasificación** sean válidas.
- Que el código planetario no esté repetido.
- Que la **población** sea mayor o igual a 0.
- Que el planeta exista antes de **buscarlo**.
- Que el planeta exista antes de **modificarlo**.
- Que el planeta exista antes de **eliminarlo**.
- Que la clasificación corresponda al rango de población.
- Que los datos numéricos sean ingresados correctamente.

**Los errores de ingreso de datos deberán ser controlados mediante:**

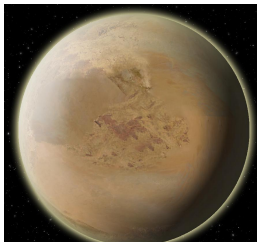
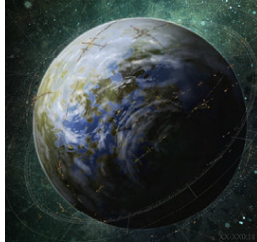
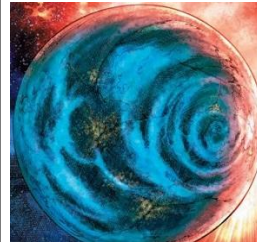
**try / catch**

El programa no deberá finalizar inesperadamente debido a un ingreso incorrecto por parte del usuario.

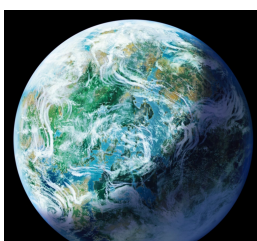
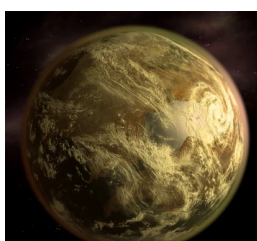
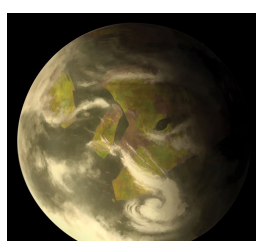
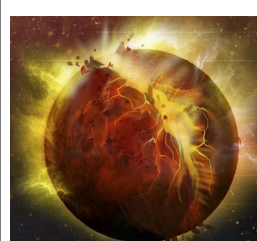
## – Ejemplos de planetas –

|                                                                                  |                                                                                   |                                                                                   |                                                                                    |                                                                                     |
|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|  |  |  |  |  |
| <b>Super Tierra</b>                                                              | <b>Necromunda</b>                                                                 | <b>Catachan</b>                                                                   | <b>Coruscant</b>                                                                   | <b>Tatooine</b>                                                                     |
| <b>Colmena</b>                                                                   | <b>Colmena</b>                                                                    | <b>Mortífero</b>                                                                  | <b>Colmena</b>                                                                     | <b>Agrícolas</b>                                                                    |
| 1.892.092.293.221                                                                | 19.092.595.220                                                                    | 620.201                                                                           | 990.925.932.213                                                                    | 320.123.155                                                                         |
| <b>Colonizado</b>                                                                | <b>Colonizado</b>                                                                 | <b>Colonizado</b>                                                                 | <b>No Colonizado</b>                                                               | <b>No Colonizado</b>                                                                |

|                                                                                   |                                                                                    |                                                                                    |                                                                                     |                                                                                      |
|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
|  |  |  |  |  |
| <b>Arrakis</b>                                                                    | <b>Macragge</b>                                                                    | <b>Cybertron</b>                                                                   | <b>Demerium</b>                                                                     | <b>Krypton</b>                                                                       |
| <b>Mortífero</b>                                                                  | <b>Colmena</b>                                                                     | <b>Forja</b>                                                                       | <b>Muerto</b>                                                                       | <b>Muerto</b>                                                                        |
| 62.340.023                                                                        | 1.209.259.322.132                                                                  | 110.289.323                                                                        | 0                                                                                   | 0                                                                                    |
| <b>No Colonizado</b>                                                              | <b>Colonizado</b>                                                                  | <b>No Colonizado</b>                                                               | <b>Colonizado</b>                                                                   | <b>No Colonizado</b>                                                                 |

|                                                                                    |                                                                                     |                                                                                     |                                                                                      |                                                                                       |
|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
|  |  |  |  |  |
| <b>Endor</b>                                                                       | <b>Helghan</b>                                                                      | <b>Marte</b>                                                                        | <b>Felucia</b>                                                                       | <b>Cadia</b>                                                                          |
| <b>Salvajes</b>                                                                    | <b>Forja</b>                                                                        | <b>Forja</b>                                                                        | <b>Agrícolas</b>                                                                     | <b>Muerto</b>                                                                         |
| 392.203                                                                            | 172.389.326                                                                         | 232.166.443                                                                         | 640.153.155                                                                          | 0                                                                                     |
| <b>No Colonizado</b>                                                               | <b>Colonizado</b>                                                                   | <b>Colonizado</b>                                                                   | <b>No Colonizado</b>                                                                 | <b>No Colonizado</b>                                                                  |

---

## – Modelo de clases –

### – Clase Colonizado –

La clase `Colonizado` deberá heredar de `Planeta` y representar aquellos planetas que se encuentran bajo un proceso de colonización o que actualmente poseen una colonia. Deberá implementar los métodos definidos por la clase abstracta `Planeta`.

### – Clase NoColonizado –

La clase `NoColonizado` deberá heredar de `Planeta` y representar aquellos planetas que aún no cuentan con una colonia establecida. Deberá implementar los métodos definidos por la clase abstracta `Planeta`.

**Importante:** tanto `Colonizado` como `NoColonizado` deberán poder clasificarse según su población.

El estado de colonización **no determina el tipo de planeta**. Por ejemplo, un planeta puede ser:

```
Colonizado
Población: 620.201
Clasificación: Mortífero
```

ó:

```
No Colonizado
Población: 0
Clasificación: Muerto
```

### – Interfaz Clasificable –

El sistema deberá incorporar una interfaz denominada:

```
Clasificable
```

Esta interfaz deberá establecer el contrato necesario para que un planeta pueda determinar su clasificación. La interfaz deberá contener un método relacionado con la clasificación del planeta, por ejemplo:

```
clasificar();
```

La clase abstracta `Planeta` deberá implementar esta interfaz. De esta manera, todo objeto que corresponda a un planeta deberá ser capaz de determinar su clasificación. La clasificación deberá depender exclusivamente de la población.

---

## – Clasificación según población –

La población determinará las posibles clasificaciones de cada planeta.

### Población igual a 0

El planeta **deberá clasificarse automáticamente como:**

### Muerto

No será necesario solicitar una clasificación adicional al usuario.

### Población entre 1 y 100.000.000

El usuario deberá seleccionar una de las siguientes clasificaciones:

1. Mortífero
2. Salvaje

No deberá ser posible seleccionar **Colmena**, **Forja** o **Agrícola** en este rango.

### Población superior a 100.000.000

El usuario deberá seleccionar una de las siguientes clasificaciones:

1. Colmena
2. Forja
3. Agrícola

No deberá ser posible seleccionar **Mortífero** o **Salvaje** en este rango.

## – Polimorfismo –

El sistema deberá utilizar **polimorfismo** mediante referencias de tipo **Planeta**.

El registro de planetas deberá permitir almacenar objetos de las clases:

Colonizado  
NoColonizado

utilizando una colección del tipo:

`ArrayList<Planeta>`

Por ejemplo:

---

```
ArrayList<Planeta> planetas;
```

La colección deberá poder contener objetos de ambas clases. Al recorrer el registro, el programa deberá poder trabajar con ellos utilizando la referencia común `Planeta`, permitiendo que cada objeto ejecute su comportamiento correspondiente.

### – Almacenamiento de información –

Los planetas deberán almacenarse utilizando un:

```
ArrayList<Planeta>
```

La clase encargada de administrar esta colección deberá ser:

```
RegistroPlanetario
```

Esta clase será responsable de administrar los planetas registrados. **Deberá incluir métodos personalizados para realizar las operaciones principales del sistema, tales como:**

- **Registrar** planeta.
- **Buscar** planeta.
- **Actualizar** planeta.
- **Eliminar** planeta.
- **Mostrar** todos los planetas.
- **Validar existencia** de un código.

El código planetario deberá ser único.

### – Menú interactivo –

El programa deberá contar con un menú que permita al usuario seleccionar las operaciones disponibles. **El menú deberá mantenerse ejecutándose mediante una estructura:**

```
“while” ó “do while”
```

hasta que el usuario seleccione la opción de salida. **El menú deberá contemplar, como mínimo:**

```
===== REGISTRO PLANETARIO =====  
1. Registrar planeta  
2. Buscar planeta  
3. Actualizar población  
4. Eliminar planeta  
5. Mostrar todos los planetas  
6. Salir
```

---

## – Entrada de datos –

La interacción con el usuario deberá realizarse utilizando:

### Scanner

El programa deberá utilizar **Scanner** para recibir:

- **Código planetario.**
- **Nombre.**
- **Población.**
- **Estado de colonización.**
- **Clasificación.**
- **Opciones del menú.**

## – Actualización de población –

Al actualizar la población de un planeta, el sistema deberá volver a evaluar su clasificación. **Por ejemplo, si un planeta posee:**

**Población:** 50.000.000  
**Clasificación:** Salvaje

y su población cambia a:

**Población:** 150.000.000

el sistema deberá solicitar una nueva clasificación entre:

1. **Colmena**
2. **Forja**
3. **Agrícola**

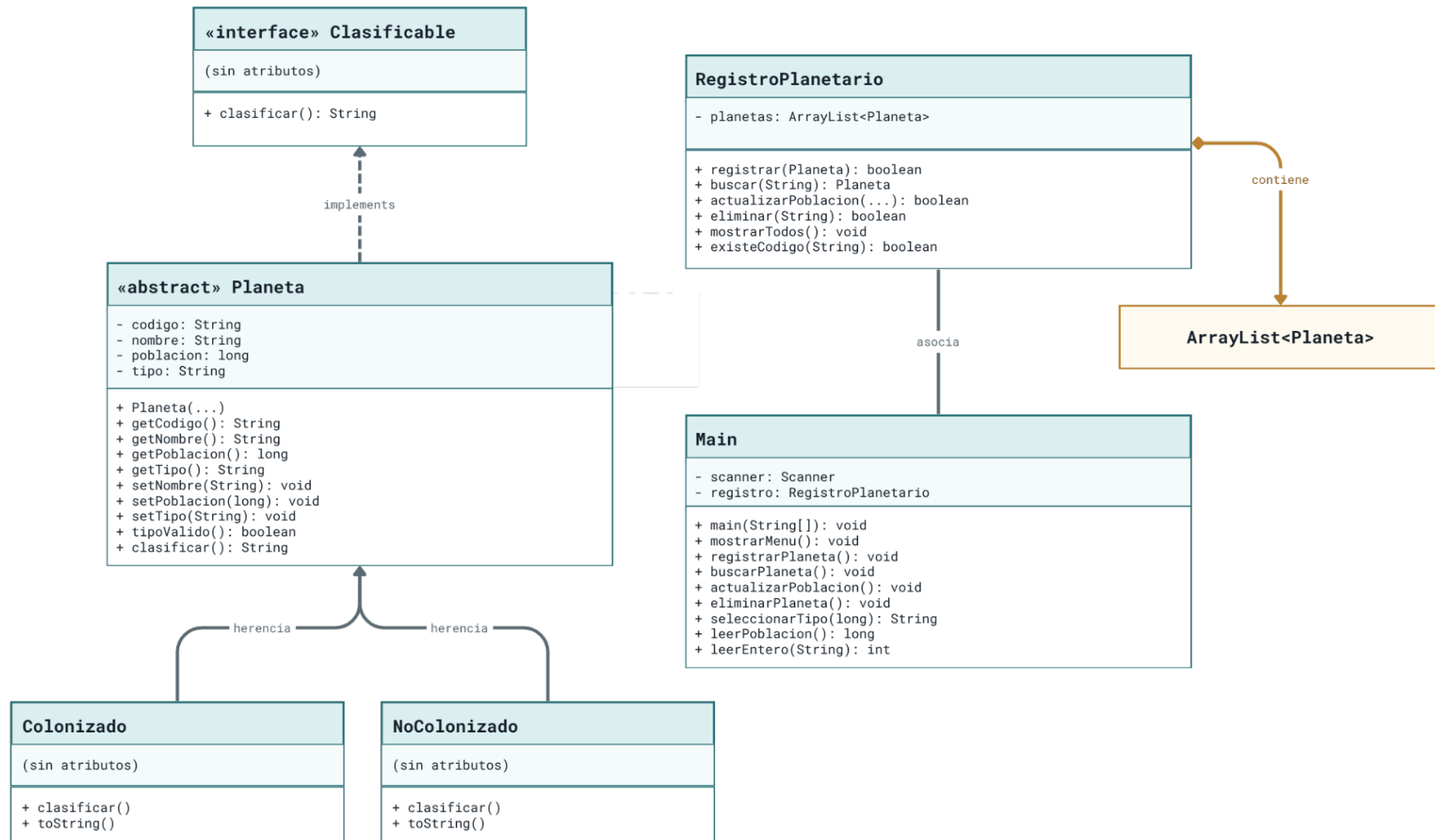
Si la población cambia a:

**0**

el planeta deberá clasificarse automáticamente como:

**Muerto**

## – Diagrama de Clases –



---

# ¡Ánimo Chicos!

---

**¡Cualquier consulta, reclamo, queja, duda que tengan no duden en contactarme!**

**¡Mándame tu ejercicio y yo te lo reviso!**

Correo Institucional: [mi.gonzalez@duocuc.cl](mailto:mi.gonzalez@duocuc.cl)